

String Programs Important programs

=====

1. Reverse a String

This program reverses a given string without using any built-in reverse methods.

```
package com.codegnan.stringexamples;

public class StringReversal {

    public static String reverseString(String str) {

        // Create an empty string to store the reversed result
        String reversed = "";

        // Iterate through the original string from the end
        for (int i = str.length() - 1; i >= 0; i--) {

            // Extract the character at the current index
            char currentChar = str.charAt(i);

            // Append the character to the reversed string
            reversed += currentChar;

        }

        return reversed;

    }

    public static void main(String[] args) {

        String input = "hello";

        System.out.println("Reversed String: " + reverseString(input));

    }

}
```

2. Check if a String is a Palindrome

```

package com.codegnan.stringexamples;

public class PalindromeCheck {

    public static boolean isPalindrome(String str) {

        // Initialize two pointers: one at the beginning and one at the end
        int left = 0;
        int right = str.length() - 1;
        // Iterate until the pointers meet
        while (left < right) {

            // Compare characters at the pointers
            if (str.charAt(left) != str.charAt(right)) {

                return false; // If characters don't match, it's not a
palindrome

            }

            // Move pointers towards the center
            left++;
            right--;

        }

        // If the loop completes without finding mismatches, it's a palindrome
        return true;

    }

    public static void main(String[] args) {

        String input1 = "racecar";
        String input2 = "hello";

        System.out.println(input1 + " is palindrome: " + isPalindrome(input1));
        System.out.println(input2 + " is palindrome: " + isPalindrome(input2));

    }

}

```

3. Check if Two Strings are Anagrams

What are Anagrams?

Anagrams are two strings (or words) that contain the same characters with the same frequency, but arranged in a different order.

Simple Definition

Two strings are anagrams if:

- They have same length
- They have same characters
- Each character appears same number of times

Examples

✓ Valid Anagrams

- "listen" → "silent"
- "eat" → "tea"
- "race" → "care"

"elbow" and "below"

"heart" and "earth"

"funeral" and "real fun"

```
public class Anagram {
```

```

public static void main(String[] args) {
    // Input strings
    String s1 = "eat";
    String s2 = "tea";

    // Step 1: Check if lengths are equal
    // If lengths are not equal, they cannot be anagrams
    if (s1.length() != s2.length()) {
        System.out.println("Not Anagram");
        return; // stop execution
    }

    // Step 2: Assume strings are anagrams initially
    boolean isAnagram = true;

    // Step 3: Traverse each character of first string
    for (int i = 0; i < s1.length(); i++) {
        // Take one character from s1
        char ch1 = s1.charAt(i);

        // Initialize counters for both strings
        int count1 = 0;
        int count2 = 0;

        // Step 4: Count occurrences of ch1 in s1
        for (int j = 0; j < s1.length(); j++) {
            if (ch1 == s1.charAt(j)) {
                count1++; // increment count in s1
            }
        }

        // Step 5: Count occurrences of same character in s2
        for (int j = 0; j < s2.length(); j++) {
            if (ch1 == s2.charAt(j)) {

```

```

        count2++; // increment count in s2
    }
}
// Step 6: Compare counts
// If counts are not equal, not an anagram
if (count1 != count2) {
    isAnagram = false; // mark as not anagram
    break; // exit loop early
}
}
// Step 7: Final result
if (isAnagram) {
    System.out.println("Anagram");
} else {
    System.out.println("Not Anagram");
}
}
}

```

Or

```

import java.util.Arrays;

public class AnagramCheck {

    public static boolean areAnagrams(String str1, String str2) {
        // Convert both strings to character arrays
    }
}

```

```

char[] charArray1 = str1.toCharArray();
char[] charArray2 = str2.toCharArray();

// Sort both character arrays alphabetically
Arrays.sort(charArray1);
Arrays.sort(charArray2);

// Compare the sorted arrays
return Arrays.equals(charArray1, charArray2);
}

public static void main(String[] args) {
    String str1 = "listen";
    String str2 = "silent";

    // Check if the strings are anagrams and print the result
    System.out.println(str1 + " and " + str2 + " are anagrams: " + areAnagrams(str1,
str2));
}
}

```

4. Remove Duplicates from a String

```

public class RemoveDuplicates {
    public static void main(String[] args) {
        // Input string

```

```

String str = "programming";
// This will store the final result (without duplicates)
String result = "";
// Step 1: Traverse each character of the original string
for (int i = 0; i < str.length(); i++) {
    // Get current character from string
    char ch = str.charAt(i);
    // This flag checks whether character already exists in result
    boolean found = false;
    // Step 2: Check if current character is already present in result
    for (int j = 0; j < result.length(); j++) {
        // Compare current character with characters in result
        if (ch == result.charAt(j)) {
            found = true; // character already exists
            break; // no need to check further
        }
    }
    // Step 3: If character is NOT found in result, add it
    if (!found) {
        result = result + ch; // append character
    }
}
// Step 4: Print final result (duplicates removed)
System.out.println("Result: " + result);
}
}

```

Input:

programming

Step-by-step:

- $p \rightarrow \text{not in result} \rightarrow \text{add} \rightarrow p$
- $r \rightarrow \text{not in result} \rightarrow \text{add} \rightarrow pr$
- $o \rightarrow \text{add} \rightarrow pro$
- $g \rightarrow \text{add} \rightarrow prog$
- $r \rightarrow \text{already exists} \times \text{skip}$
- $a \rightarrow \text{add} \rightarrow proga$
- $m \rightarrow \text{add} \rightarrow program$
- $m \rightarrow \text{duplicate} \times \text{skip}$
- $i \rightarrow \text{add} \rightarrow programi$
- $n \rightarrow \text{add} \rightarrow programin$
- $g \rightarrow \text{duplicate} \times \text{skip}$

5. Print duplicates in string

```
public class PrintDuplicateCharacters {  
    public static void main(String[] args) {  
        // Input string  
        String str = "Hello";  
        // Step 1: Traverse each character in the string  
        for (int i = 0; i < str.length(); i++) {  
            // Count occurrences of current character  
            int count = 0;
```



```

// This flag is used to avoid printing duplicates again
boolean alreadyPrinted = false;

// Step 2: Check if character is already processed earlier
for (int k = 0; k < i; k++) {
    if (str.charAt(i) == str.charAt(k)) {
        alreadyPrinted = true; // already handled
        break;
    }
}

// If already printed, skip this character
if (alreadyPrinted) {
    continue;
}

// Step 3: Count how many times this character appears
for (int j = 0; j < str.length(); j++) {
    if (str.charAt(i) == str.charAt(j)) {
        count++;
    }
}

// Step 4: If count > 1, it is a duplicate
if (count > 1) {
    System.out.println(str.charAt(i));
}
}
}
}

```

6. Program to count the duplicate characters in a string

```
package com.codegnan.stringexamples;

public class DuplicateCharacterCount {

    public static int countDuplicates(String str) {
        int count = 0;
        for (int i = 0; i < str.length(); i++) {
            for (int j = i + 1; j < str.length(); j++) {
                if (str.charAt(i) == str.charAt(j)) {
                    count++;
                    break; // Avoid counting duplicates of the same character multiple
times
                }
            }
        }
        return count;
    }

    public static void main(String[] args) {
        String str = "helloappmmzzrro";
        int duplicateCount = countDuplicates(str);
        System.out.println("Number of duplicates: " + duplicateCount); // Output: 4
    }
}
```

7. Vowels and consonants in a given string

```
=====

package com.codegnan.oopprogramming;

public class VowelConsonant {

    public static void main(String[] args) {

        // Input string
        String str = "JavaProgramming";

        // Call method
        printVowelsAndConsonants(str);
    }

    // Method to print vowels and consonants
    public static void printVowelsAndConsonants(String str) {

        // Strings to store vowels and consonants
        String vowels = "";
        String consonants = "";

        // Step 1: Traverse each character in the string
        for (int i = 0; i < str.length(); i++) {

            char ch = str.charAt(i);

            // Step 2: Check if character is alphabet
            if ((ch >= 'A' && ch <= 'Z') || (ch >= 'a' && ch <= 'z')) {

                // Step 3: Check if vowel
                if (ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U' ||
                    ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {

                    // Add to vowels string
                    vowels = vowels + ch;

                } else {

                    // Otherwise it is consonant
                    consonants = consonants + ch;
                }
            }
        }
    }
}
```

```

        }
    }
}

// Step 4: Print results

System.out.println("Vowels: " + vowels);

System.out.println("Consonants: " + consonants);

}

}

```

8. Vowels and consonant counts in a given string

```

package com.codegnan.stringexamples;

public class VowelConsonantCount {
    public static void main(String[] args) {
        // Input string
        String str = "JavaProgramming";

        // Call method to count vowels and consonants
        countVowelsAndConsonants(str);
    }

    // Method to count vowels and consonants
    public static void countVowelsAndConsonants(String str) {
        int vowels = 0;
        int consonants = 0;

        // Step 1: Traverse each character in string
        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);

```

```

// Step 2: Check if character is an alphabet
if ((ch >= 'A' && ch <= 'Z') || (ch >= 'a' && ch <= 'z')) {
    // Step 3: Check if character is a vowel
    if (ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U' ||
        ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {
        vowels++; // increase vowel count
    } else {
        consonants++; // otherwise it is a consonant
    }
}
}

// Step 4: Print results
System.out.println("Vowels: " + vowels);
System.out.println("Consonants: " + consonants);
}
}

```

9.. Count Occurrences of a Character in a String

=====

```

package com.codegnan.stringexamples;

public class CountCharacterOccurrences {

    public static int countOccurrences(String str, char c) {
        // 1. Initialize a counter to keep track of occurrences
        int count = 0;

        // 2. Iterate through each character in the string
        for (int i = 0; i < str.length(); i++) {

```

```

        // 3. Check if the current character matches the target character
        if (str.charAt(i) == c) {
            // 4. If a match is found, increment the count
            count++;
        }
    }

    // 5. Return the total count of occurrences
    return count;
}

public static void main(String[] args) {
    // 6. Define the input string and the target character
    String inputString = "hello world";
    char targetChar = 'l';

    // 7. Call the countOccurrences method to get the result
    int occurrences = countOccurrences(inputString, targetChar);

    // 8. Print the result to the console
    System.out.println("Occurrences of " + targetChar + " in " + inputString + ": " +
occurrences);
}
}

```

11. Count Occurrence Of each character in a given String

```

package com.codegnan.oopprogramming;

public class Frequency {
    public static void main(String[] args) {
        // Input string
    }
}

```

```

String str = "hello";

// Call method to calculate and print frequency
printFrequency(str);
}

// Method to calculate and print frequency of each character
public static void printFrequency(String str) {
    // Step 1: Traverse each character in the string
    for (int i = 0; i < str.length(); i++) {
        // Variable to count occurrences of current character
        int count = 0;

        // Flag to check whether character is already processed
        boolean visited = false;

        // Step 2: Check if this character appeared earlier
        // If yes, skip it to avoid duplicate printing
        for (int k = 0; k < i; k++) {
            if (str.charAt(i) == str.charAt(k)) {
                visited = true; // already counted
                break;
            }
        }

        // If already visited, skip this iteration
        if (visited) {
            continue;
        }

        // Step 3: Count how many times this character appears
        for (int j = 0; j < str.length(); j++) {
            if (str.charAt(i) == str.charAt(j)) {
                count++; // increase count
            }
        }
    }
}

```

```

    }
}
// Step 4: Print character and its frequency
System.out.println(str.charAt(i) + ":" + count);
}
}
}

```

Or

```

package com.codegnan.stringexamples;
import java.util.HashMap;
import java.util.Map;
public class CharacterCount {
    public static void countCharacterOccurrences(String str) {
        // 1. Create a HashMap to store character frequencies
        Map<Character, Integer> charCountMap = new HashMap<>();
        // 2. Iterate through each character in the string
        for (char c : str.toCharArray()) {
            // 3. Get the current count of the character from the map
            int count = charCountMap.getOrDefault(c, 0);
            // 4. Update the count of the character in the map
            charCountMap.put(c, count + 1);
        }
        // 5. Print the character frequencies
        for (Map.Entry<Character, Integer> entry : charCountMap.entrySet()) {
            System.out.println(entry.getKey() + ":" + entry.getValue());
        }
    }
}

```



```

    }
    public static void main(String[] args) {
        String str = "hello world";
        countCharacterOccurrences(str);
    }
}

```

12. Remove white space

=====

```

package com.codegnan.stringexamples;

public class RemoveWhitespace {
    public static String removeWhitespace(String str) {
        // 1. Create an empty string to store the result
        String result = "";
        // 2. Iterate through each character in the string
        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            // 3. Check if the character is not a space, tab, or newline
            if (ch != ' ' && ch != '\t' && ch != '\n') {
                // 4. Append the character to the result string
                result += ch;
            }
        }
        // 5. Return the string without whitespace
        return result;
    }

    public static void main(String[] args) {
        String str = " This is a string with whitespace ";
    }
}

```

```

String result = removeWhitespace(str);

System.out.println("Original string: " + str);

System.out.println("String without whitespace: " + result);

}

}

```

Or

```

package com.codegnan.stringexamples;

public class RemoveWhitespace {

    public static String removeWhitespace(String str) {
        // 1. Create an empty string to store the result
        String result = "";

        // 2. Iterate through each character in the string
        for (int i = 0; i < str.length(); i++) {
            // 3. Get the current character
            char ch = str.charAt(i);

            // 4. Check if the character is not a whitespace character
            if (!Character.isWhitespace(ch)) {
                // 5. Append the character to the result string
                result += ch;
            }
        }

        // 6. Return the string without whitespace
        return result;
    }

    public static void main(String[] args) {
        String str = " This is a string with whitespace ";
        String result = removeWhitespace(str);
    }
}

```

```
System.out.println("Original string: " + str);  
System.out.println("String without whitespace: " + result);  
}  
}
```

Question 13: . Word Count (No split)

```
package com.codegnan.oopprogramming;  
public class WordCount {  
    public static void main(String[] args) {  
        // Input string  
        String str = "Java is easy to learn";  
        // Call method to count words  
        countWords(str);  
    }  
    // Method to count words in a string  
    public static void countWords(String str) {  
        int count = 0;  
        // Step 1: Count number of spaces  
        for (int i = 0; i < str.length(); i++) {  
            if (str.charAt(i) == ' ') {  
                count++; // each space separates words  
            }  
        }  
        // Step 2: If string is not empty, words = spaces + 1  
        if (str.length() > 0) {  
            count++;  
        }  
    }  
}
```

```

    }
    // Step 3: Print result
    System.out.println("Word Count: " + count);
}
}

```

13. First Non-Repeated Character

```

public class FirstNonRepeated {
    public static void main(String[] args) {
        // Input string
        String str = "aabbcdde";
        // Call method to find first non-repeated character
        findFirstNonRepeated(str);
    }
    // Method to find first non-repeated character
    public static void findFirstNonRepeated(String str) {
        // Step 1: Traverse each character
        for (int i = 0; i < str.length(); i++) {
            int count = 0;
            // Step 2: Count occurrences of current character
            for (int j = 0; j < str.length(); j++) {
                if (str.charAt(i) == str.charAt(j)) {
                    count++;
                }
            }
            // Step 3: If count is 1 → first non-repeated character found

```

```

        if (count == 1) {
            System.out.println("First Non-Repeated: " + str.charAt(i));
            break; // stop after first match
        }
    }
}
}
}

```

13.) Java program to print each letter twice from a given string

```

package com.codegnan.stringprograms;
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String input = scanner.nextLine();
        String doubledString = doubleCharacters(input);
        System.out.println("Doubled characters: " +
doubledString);
    }
    public static String doubleCharacters(String str) {
        StringBuilder doubled = new StringBuilder();
        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            doubled.append(ch).append(ch); // Append each
character
        }
        return doubled.toString();
    }
}

```

(or)

```
public class PrintTwice {  
    public static void main(String[] args) {  
        // Input string  
        String str = "Java";  
        // Call method  
        printEachTwice(str);  
    }  
    // Method to print each character twice  
    public static void printEachTwice(String str) {  
        // Step 1: Traverse each character in the  
        string  
        for (int i = 0; i < str.length(); i++) {  
            // Get character at index i  
            char ch = str.charAt(i);  
            // Step 2: Print character two times  
            System.out.print(ch); // first time  
            System.out.print(ch); // second time  
        }  
    }  
}
```

Output:-

Enter a string: hello

Doubled characters: hheelllloo

14.) Java program to swap two string without using 3rd variable

```
-----  
package com.codegnan.strings;  
import java.util.Scanner;  
public class StringSwapper {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
        System.out.print("Enter First String : ");  
        String str1 = scanner.nextLine();  
        System.out.print("Enter second String : ");  
        String str2 = scanner.nextLine();  
        System.out.println("Beffore Swapping : str1 = " + str1 + " ,  
str2 " + str2);  
        // concatenate str1 and str2 and store in str1  
        str1 = str1 + str2; // malli+arjun==>malliarjun  
        // extract the original str1 by taking substring from 0  
to length of str1-  
        // length of str2  
        str2 = str1.substring(0, str1.length() - str2.length()); // 0,4  
        // extract the original str2 by taking substring fron  
length of str2 to end.  
        str1 = str1.substring(str2.length());  
        // display the values after swapping  
        System.out.println("after Swapping : str1 = " + str1 + " ,  
str2 " + str2);  
    }  
}
```

Output

Enter first string: Hello

Enter second string: World

Before swapping: str1 = Hello, str2 = World

After swapping: str1 = World, str2 = Hello

15.) Java program to gives Output: a2b2c3d2 for the Input String Str = "aabbccdd"

```
package com.codegnan.stringprograms;
import java.util.Scanner;
public class CharacterCompressor {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        // Prompt user for input
        System.out.print("Enter a string: ");
        String input = scanner.nextLine();
        // Get the compressed result
        String output = compressRepeatedCharacters(input);
        // Display the output
        System.out.println("Output: " + output);
        scanner.close();
    }
    public static String compressRepeatedCharacters(String
str) {
        StringBuilder result = new StringBuilder();
        int count = 1;
        for (int i = 0; i < str.length(); i++) {
            // If the next character is the same, increase the
count
            if (i + 1 < str.length() && str.charAt(i) == str.charAt(i
+ 1)) {
                count++;
            } else {
                // Append the current character and its
count
                result.append(str.charAt(i)).append(count);
                count = 1; // Reset count for the next
character
            }
        }
    }
}
```



```
        return result.toString();
    }
}
```

(or)

```
public class StringCompression {
    public static void main(String[] args) {
        // Input string
        String str = "aabbcccd";
        // Call method to compress string
        compressString(str);
    }
    // Method to compress string (character +
    count)
    public static void compressString(String str) {
        // Step 1: Initialize result string
        String result = "";
        // Step 2: Loop through string
        for (int i = 0; i < str.length(); i++) {
            // Take current character
            char current = str.charAt(i);
            // Count occurrences of current character
            int count = 1;
            // Step 3: Check next characters
            (consecutive same characters)
            while (i + 1 < str.length() && str.charAt(i) ==
            str.charAt(i + 1)) {
                count++; // increase count
            }
        }
    }
}
```

```

        i++;    // move to next index
    }
    // Step 4: Add character and its count to
result
    result = result + current + count;
}
// Step 5: Print compressed string
System.out.println(result);
}
}

```

Output:

Enter a string: aabbccdd

Output: a2b2c3d2

16.) Java program to gives two Output: "abcde", "ABCDE" for the Input String Str = "aBACbcEDed"

```
package com.codegnan.stringprograms;
```

```
import java.util.Scanner;
```

```
import java.util.Scanner;
```

```
/**
```

```
 * This program reads a string from the user and separates the
lowercase and
```

```
 * uppercase characters into different outputs.
```

```
*/
```

```
public class CharacterCompressor {
```

```
    public static void main(String[] args) {
```

```
        Scanner scanner = new Scanner(System.in);
```

```

        // Prompt user for input
        System.out.print("Enter a string: ");
        String input = scanner.nextLine();

        // Display original string
        System.out.println("Original String is: " + input);

        // Call method to separate and print characters
        separateCharactersByCase(input);

        scanner.close();
    }

    /**
     * Separates lowercase and uppercase characters from the
input string and prints
     * them.
     *
     * @param input The input string
     */
    public static void separateCharactersByCase(String input)
    {
        StringBuilder lowerCase = new StringBuilder();
        StringBuilder upperCase = new StringBuilder();

        for (char ch : input.toCharArray()) {
            if (Character.isLowerCase(ch)) {
                lowerCase.append(ch);
            } else{
                if (Character.isUpperCase(ch)) {
                    upperCase.append(ch);
                }
            }
        }
    }

```

```

        System.out.println("Output in lowercase: " + lowerCase);
        System.out.println("Output in uppercase: " +
upperCase);
    }
}

```

(or)

```

public class SeparateCases {
    public static void main(String[] args) {
        // Input string
        String str = "aBACbcEDed";
        // Call method
        separateCharacters(str);
    }
    // Method to separate lowercase and
uppercase characters (without duplicates)
    public static void separateCharacters(String
str) {
        // Strings to store results
        String lowerResult = "";
        String upperResult = "";
        // Step 1: Traverse each character in the
string
        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            // Step 2: Check if lowercase letter
            if (ch >= 'a' && ch <= 'z') {
                boolean found = false;
                // Step 3: Check if already added in
lowerResult

```

```

        for (int j = 0; j < lowerResult.length(); j++) {
            if (ch == lowerResult.charAt(j)) {
                found = true;
                break;
            }
        }
        // Step 4: If not duplicate, add to result
        if (!found) {
            lowerResult = lowerResult + ch;
        }
    }
    // Step 5: Check if uppercase letter
    else if (ch >= 'A' && ch <= 'Z') {
        boolean found = false;
        // Step 6: Check if already added in
upperResult
        for (int j = 0; j < upperResult.length(); j++) {
            if (ch == upperResult.charAt(j)) {
                found = true;
                break;
            }
        }
        // Step 7: If not duplicate, add to result
        if (!found) {
            upperResult = upperResult + ch;
        }
    }
}

```

```

// Step 8: Print results
System.out.println("Lowercase: " + lowerResult);
System.out.println("Uppercase: " +
upperResult);
    }
}

```

17.) Java program to gives two Output: "Subburaj", "123" for the Input String Str = "Subbu123raj"

```

package com.codegnan.stringprograms;
import java.util.Scanner;
/**
 * This program separates alphabetic and numeric characters
 * from a given input
 * string using an alternate if-else format.
 */
public class AlphaNumericSeparator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        // Prompt user to enter a string
        System.out.print("Enter a string: ");
        String input = scanner.nextLine();
        // Display the original string
        System.out.println("Original String is: " + input);
        // Call the method to separate and print results
        separateAlphaAndNumeric(input);
        scanner.close();
    }
    /**
     * Separates letters and digits from the input string using
     * nested else-if
     * logic.
     *

```

```

    * @param input The input string from which to extract
    letters and numbers
    */
    public static void separateAlphaAndNumeric(String input)
    {
        StringBuilder alphaPart = new StringBuilder();
        StringBuilder numericPart = new StringBuilder();
        for (char ch : input.toCharArray()) {
            if (Character.isLetter(ch)) {
                alphaPart.append(ch);
            } else {
                if (Character.isDigit(ch)) {
                    numericPart.append(ch);
                }
            }
        }
        // Output the results
        System.out.println("Output in Alpha: " +
alphaPart.toString());
        System.out.println("Output in Numeric: " +
numericPart.toString());
    }
}

```

(or)

```

package com.codegnan.oopprogramming;
public class SeparateAlphaDigits {
    public static void main(String[] args) {
        // Input string
        String str = "Subbu123raj";
        // Call method
        separateAlphaAndDigits(str);
    }
}

```

```

// Method to separate alphabets and digits
public static void
separateAlphaAndDigits(String str) {
    // Strings to store results
    String letters = "";
    String digits = "";
    // Step 1: Traverse each character in the
string
    for (int i = 0; i < str.length(); i++) {
        // Get current character
        char ch = str.charAt(i);
        // Step 2: Check if character is an alphabet
(A-Z or a-z)
        if ((ch >= 'A' && ch <= 'Z') || (ch >= 'a' && ch <=
'z')) {
            // Add to letters string
            letters = letters + ch;
        }
        // Step 3: Check if character is a digit (0-9)
        else if (ch >= '0' && ch <= '9') {
            // Add to digits string
            digits = digits + ch;
        }
    }
    // Step 4: Print results
    System.out.println("Alphabets: " + letters);
    System.out.println("Digits: " + digits);
}

```


}

Enter a string: Subbu123raj

Output in Alpha: Subburaj

Output in Numeric: 123

Question-1

Write a program to compare two strings lexicographically and return -1, 0, or 1.

It compares two strings alphabetically (case-sensitive) and returns:

-1 if first comes before second

0 if equal

1 if first comes after second

Input Format

String 1

String 2

Output Format

Print -1, 0, or 1

Sample Test Case 1

Input

apple

banana

Output

-1

Explanation

"apple" < "banana" so output is -1.

Sample Test Case 2

Input

cat
cat

Output

0

Explanation

Both strings are equal.

Sample Test Case 3

Input

zebra
apple

Output

1

Explanation

"zebra" > "apple".

Java Code

```
package  
com.codegnan.stringpracticeexamples;  
import java.util.Scanner; // Import Scanner  
public class Main { // Declare main class  
    public static void main(String[] args) { //  
Main method
```

```

Scanner scanner = new
Scanner(System.in); // Scanner object to
read input
    String str1 = scanner.nextLine(); //
Read first string
    String str2 = scanner.nextLine(); //
Read second string
    int result = str1.compareTo(str2); //
Compare lexicographically
    if (result < 0) { // If str1 comes before
str2
        System.out.println(-1);
    } else if (result == 0) { // If both strings
are equal
        System.out.println(0);
    } else { // If str1 comes after str2
        System.out.println(1);
    }
}
}

```

Question-2

Write a program to concatenate two strings.

Input Format

String 1

String 2

Output Format

Print concatenated string.

Sample Test Case 1

Input

Good

Morning

Output

GoodMorning

Explanation

Strings are joined without space.

Sample Test Case 2

Input

Java

Code

Output

JavaCode

Explanation

Concatenation of two strings.

Sample Test Case 3

Input

A

B

Output
AB

Explanation
Simple concatenation.

Java Code

```
package
com.codegnan.stringpracticeexamples;
import java.util.Scanner; // Import Scanner
public class Main { // Declare main class
    public static void main(String[] args) { //
Main method
        Scanner scanner = new
Scanner(System.in); // Create Scanner
        String str1 = scanner.nextLine(); //
Read first input string
        String str2 = scanner.nextLine(); //
Read second input string
        String result = str1 + str2; //
Concatenate strings
        System.out.println(result); // Print
result
    }
}
```

Question-3

Write a program to concatenate two strings with a given separator.

Input Format

String 1

String 2

Separator (1 character)

Output Format

Print concatenated string with separator.

Sample Test Case 1

Input

Apple

Orange

Output

Apple-Orange

Explanation

Strings joined using '-' separator.

Sample Test Case 2

Input

Java

Python

Output

Java#Python

Explanation

Separator placed in between.

Sample Test Case 3

Input

Cat

Dog

,

Output

Cat,Dog

Explanation

Comma used as separator.

Java Code

package

com.codegnan.stringpracticeexamples;

import java.util.Scanner; // Import Scanner

public class Main { // Main class

public static void main(String[] args) { //

Main method

 Scanner scanner = **new**

Scanner(System.**in**); // Scanner object

 String str1 = scanner.nextLine(); //

Read first string

```
        String str2 = scanner.nextLine(); //
Read second string
        String separator = scanner.nextLine();
// Read separator
        String result = str1 + separator + str2;
// Build final string
        System.out.println(result); // Print
result
    }
}
```

Question-4

Write a program to count occurrences of a character in a string between given indices.

Input Format

A string

A character

Start index

End index

Output Format

Print count of occurrences.

Sample Test Case 1

Input

Mississippi

s
2
8

Output
4

Explanation
's' appears 4 times between index 2 and 7.

Sample Test Case 2

Input
Programming
r
1
7

Output
2

Explanation
'r' appears twice.

Sample Test Case 3

Input
Hello
l
1
4

Output
2

Explanation
'l' occurs 2 times.

Java Code

```
package
com.codegnan.stringpracticeexamples;
import java.util.Scanner; // Import Scanner
public class Main { // Declare class
    public static void main(String[] args) {
        Scanner scanner = new
Scanner(System.in); // Scanner object
        String str = scanner.nextLine(); //
Input string
        char target =
scanner.nextLine().charAt(0); // Read target
character
        int start = scanner.nextInt(); // Read
start index
        int end = scanner.nextInt(); // Read
end index
        int count = 0; // Counter
        for (int i = start; i < end; i++) { // Loop
through range
            if (str.charAt(i) == target) { //
Compare characters
                count++; // Increase count
            }
        }
    }
}
```

```
        System.out.println(count); // Print
result
    }
}
```

Question-5

Write a program to extract a substring using start and end indices.

Input Format

A string

Start index

End index

Output Format

Print extracted substring.

Sample Test Case 1

Input

HelloWorld

2

7

Output

lloWo

Explanation

Substring from index 2 to 6.

Sample Test Case 2

Input
Programming
3
8

Output
gramm

Explanation
Characters from index 3 to 7.

Sample Test Case 3

Input
JavaCode
4
8

Output
Code

Explanation
Extracted correctly.

Java Code

```
package
com.codegnan.stringpracticeexamples;
import java.util.Scanner; // Import Scanner
public class Main { // Declare class
    public static void main(String[] args) {
        Scanner scanner = new
Scanner(System.in); // Scanner
```

```

        String input = scanner.nextLine(); //
Read entire string
        int start = scanner.nextInt(); // Start
index
        int end = scanner.nextInt(); // End
index
        String result = input.substring(start,
end); // Extract substring
        System.out.println(result); // Print
output
    }
}

```

Question-6

Write a program to replace the last occurrence of a substring in a StringBuilder.

Input Format

- A main string
- A substring to replace
- A replacement string

Output Format

- Print updated string.

Sample Test Case 1

Input

Hello Hello World

Hello

Earth

Output

Hello Earth World

Explanation

The last "Hello" is replaced with "Earth".

Sample Test Case 2

Input

JavaCodeJava

Java

Python

Output

JavaCodePython

Explanation

Only last "Java" is replaced.

Sample Test Case 3

Input

abc abc abc

abc

xyz

Output

abc abc xyz

Explanation

Last occurrence replaced.

Java Code

```
package
com.codegnan.stringpracticeexamples;
import java.util.Scanner; // Import Scanner
public class Main { // Declare main class
    public static void main(String[] args) { //
Main method
        Scanner scanner = new
Scanner(System.in); // Create scanner
        StringBuilder sb = new
StringBuilder(scanner.nextLine()); // Read full
string
        String target = scanner.nextLine(); //
Read substring to replace
        String replacement =
scanner.nextLine(); // Read replacement
        int lastIndex = sb.lastIndexOf(target);
// Find last occurrence
        if (lastIndex != -1) { // If substring
exists
            sb.replace(lastIndex, lastIndex +
target.length(), replacement); // Replace it
        }
        System.out.println(sb.toString()); //
Print updated string
    }
```

```
}  
}
```

Question-7

Write a program to append a string and a number using StringBuilder.

Input Format

A string

An integer

Output Format

Print string followed by number.

Sample Test Case 1

Input

Level

5

Output

Level5

Explanation

Appended 5 after the word.

Sample Test Case 2

Input

Score

100

Output
Score100

Explanation
Simple append.

Sample Test Case 3

Input
Day
1

Output
Day1

Explanation
String + integer.

Java Code

```
package
com.codegnan.stringpracticeexamples;
import java.util.Scanner; // Import Scanner
public class Main { // Main class
    public static void main(String[] args) { //
Main method
        Scanner scanner = new
Scanner(System.in); // Scanner
        String text = scanner.nextLine(); //
Read string
```

```
        int number = scanner.nextInt(); //
Read number
        StringBuilder sb = new
StringBuilder(text); // Create builder with
text
        sb.append(number); // Append
integer
        System.out.println(sb.toString()); //
Print result
    }
}
```

Question-8

Write a program to delete the first character from a StringBuilder.

Input Format

A string

Output Format

Print string after deleting the first character.

Sample Test Case 1

Input

Hello

Output
ello

Explanation
Removed first 'H'.

Sample Test Case 2

Input
Java

Output
ava

Explanation
Deleted first character.

Sample Test Case 3

Input
Programming

Output
rogramming

Explanation
Starts from second character.

Java Code

```
package  
com.codegnan.stringpracticeexamples;  
import java.util.Scanner; // Import Scanner  
public class Main { // Class  
    public static void main(String[] args) {
```

```

        Scanner scanner = new
Scanner(System.in); // Scanner
        String input = scanner.nextLine(); //
Read input string
        StringBuilder sb = new
StringBuilder(input); // Build string
        if (sb.length() > 0) { // Check
non-empty
            sb.deleteCharAt(0); // Delete first
character
        }
        System.out.println(sb.toString()); //
Print result
    }
}

```

Question-9

Write a program to compress a string using character counts.

(Example: aabcc → a2b1c2)

Return the original string if compressed version is longer;

If equal length, return compressed.

Input Format

A lowercase string

Output Format

Print compressed string.

Sample Test Case 1

Input

aabcccccaaa

Output

a2b1c5a3

Explanation

Repeated counts appended.

Sample Test Case 2

Input

abc

Output

abc

Explanation

Compressed = a1b1c1 (length 6 > 3), so original returned.

Sample Test Case 3

Input

aaa

Output

a3

Explanation

Compressed shorter, so returned.

Java Code

```
package
com.codegnan.stringpracticeexamples;
import java.util.Scanner; // Import Scanner
public class Main { // Main class
    public static String
compressString(String s) { // Compression
method
    StringBuilder compressed = new
StringBuilder(); // Store compressed
    int count = 1; // Count of characters
    for (int i = 1; i < s.length(); i++) { // Loop
through string
        if (s.charAt(i) == s.charAt(i - 1)) { // If
same char repeats
            count++;
        } else { // If new character
            compressed.append(s.charAt(i
- 1)).append(count); // Append char + count
            count = 1; // Reset counter
        }
    }

    compressed.append(s.charAt(s.length() -
1)).append(count); // Append last group
```

```

        return compressed.length() <=
s.length() ? compressed.toString() : s; //
Return shorter/equal
    }
    public static void main(String[] args) { //
Main method
        Scanner scanner = new
Scanner(System.in); // Scanner
        String input = scanner.nextLine(); //
Read input

System.out.println(compressString(input)); //
Print result
    }
}

```

Question-10

Write a program to remove duplicate characters, keeping only first occurrences.

Input Format

A string

Output Format

Print string without duplicates

Sample Test Case 1

Input
hello

Output
helo

Explanation
Keep first 'l'.

Sample Test Case 2

Input
aabbcc

Output
abc

Explanation
Remove repeated letters.

Sample Test Case 3

Input
adding

Output
ading

Explanation
'D' removed only once.

Java Code

```
package  
com.codegnan.stringpracticeexamples;
```



```
import java.util.Scanner; // Import Scanner
import java.util.HashSet; // Import HashSet
public class Main { // Class
    public static String
removeDuplicates(String s) { // Method to
remove duplicates
    StringBuilder result = new
StringBuilder(); // Result holder
    HashSet<Character> seen = new
HashSet<>(); // Track seen chars
    for (char c : s.toCharArray()) { // Loop
through string
        if (!seen.contains(c)) { // If char not
yet added
            seen.add(c); // Mark as seen
            result.append(c); // Append
char
        }
    }
    return result.toString(); // Return
cleaned string
}
    public static void main(String[] args) { //
Main method
```

```
Scanner scanner = new
Scanner(System.in); // Scanner
String input = scanner.nextLine(); //
Read input

System.out.println(removeDuplicates(input));
// Print output
    }
}
```

Question-11

Write a program to find the longest word in a string.

Words are separated by spaces.

If multiple words have the same length, return the first one.

Input Format

A string containing words separated by spaces

Output Format

Print the longest word.

Sample Test Case 1

Input

I love coding

Output

coding

Explanation

"coding" is longest.

Sample Test Case 2

Input

a b c

Output

a

Explanation

All same length → return first.

Sample Test Case 3

Input

hello world

Output

hello

Explanation

Both 5 letters → first returned.

Java Code

```
package  
com.codegnan.stringpracticeexamples;  
import java.util.Scanner; // Import Scanner  
public class Main { // Declare class
```

```
    public static String
findLongestWord(String s) { // Method to
find longest word
        String[] words = s.split(" "); // Split by
spaces
        String longest = ""; // To store longest
word
        for (String word : words) { // Loop
through words
            if (word.length() > longest.length()) {
// Compare lengths
                longest = word; // Update
longest
            }
        }
        return longest; // Return longest word
    }

    public static void main(String[] args) { //
Main method
        Scanner scanner = new
Scanner(System.in); // Scanner
        String input = scanner.nextLine(); //
Read string input
```

```
System.out.println(findLongestWord(input));  
// Print longest word  
    }  
}
```

Question-12

Write a program to find the length of a string excluding all spaces.

Input Format

A string (may contain spaces)

Output Format

Print length excluding spaces.

Sample Test Case 1

Input

hello world

Output

10

Explanation

"helloworld" has 10 chars.

Sample Test Case 2

Input

a b c

Output
3

Explanation

Spaces removed → abc.

Sample Test Case 3

Input
hi there

Output
7

Explanation

"hithere" length = 7.

Java Code

```
package
com.codegnan.stringpracticeexamples;
import java.util.Scanner; // Import Scanner
public class Main { // Class
    public static void main(String[] args) { //
Main method
        Scanner scanner = new
Scanner(System.in); // Scanner
        String input = scanner.nextLine(); //
Read entire string
```

```
        int lengthWithoutSpaces =  
input.replace(" ", "").length(); // Remove spaces  
and count length
```

```
System.out.println(lengthWithoutSpaces); //  
Print result  
    }  
}
```

Question-13

Write a program to capitalize the first letter of a string and keep the rest unchanged.

Input Format

A lowercase or uppercase string

Output Format

Print updated string.

Sample Test Case 1

Input

hello

Output

Hello

Explanation

First letter capitalized.

Sample Test Case 2

Input

abc

Output

Abc

Explanation

Only first letter changes.

Sample Test Case 3

Input

world

Output

World

Explanation

Same logic.

Java Code

```
package
com.codegnan.stringpracticeexamples;
import java.util.Scanner; // Import Scanner
public class Main { // Main class
    public static void main(String[] args) {
        Scanner scanner = new
Scanner(System.in); // Scanner
```



```

        String input = scanner.nextLine(); //
Read string
        if (input.length() == 0) { // If empty
string
                System.out.println(""); // Print
empty
        } else {
                String result = input.substring(0,
1).toUpperCase() + input.substring(1); //
Capitalize first letter
                System.out.println(result); // Print
output
        }
    }
}

```

Question-14

Write a program to remove all spaces from a string.

Input Format

A string with spaces

Output Format

Print string without spaces.

Sample Test Case 1

Input

hello world

Output

helloworld

Explanation

Spaces removed.

Sample Test Case 2

Input

a b c

Output

abc

Explanation

All spaces removed.

Sample Test Case 3

Input

hi there

Output

hithere

Explanation

Spaces deleted.

Java Code

```
package
com.codegnan.stringpracticeexamples;
import java.util.Scanner; // Import Scanner
public class Main { // Main class
    public static void main(String[] args) {
        Scanner scanner = new
Scanner(System.in); // Scanner
        String input = scanner.nextLine(); //
Read string
        String result = input.replace(" ", ""); //
Replace spaces
        System.out.println(result); // Print
result
    }
}
```

Question-15

Write a program to count the number of vowels (a, e, i, o, u) in a string.

String contains only lowercase English letters.

Input Format

A lowercase string

Output Format

Print vowel count.

Sample Test Case 1

Input
apple

Output
2

Explanation
'a' and 'e'.

Sample Test Case 2

Input
aeiou

Output
5

Explanation
All vowels.

Sample Test Case 3

Input
hello

Output
2

Explanation
'e' and 'o'.

Java Code

```
package
com.codegnan.stringpracticeexamples;
import java.util.Scanner; // Import Scanner
public class Main { // Class
    public static void main(String[] args) {
        Scanner scanner = new
Scanner(System.in); // Scanner
        String input = scanner.nextLine(); //
Read string
        int count = 0; // Counter
        for (char ch : input.toCharArray()) { //
Loop through characters
            if (ch == 'a' || ch == 'e' || ch == 'i' || ch
== 'o' || ch == 'u') { // Check vowel
                count++; // Increase count
            }
        }
        System.out.println(count); // Print
result
    }
}
```